

Vol 14 Chapter 7 — AC Graph as Theorem Network

Keeper (author pass — deep math/physics revision)

2026-05-24 Sunday

Chapter 7 — AC Graph as Theorem Network

Extended-reach chapter (May 2026), identification tier. Stamped 2026-09-13 (Keeper, K1895): the same five integers appear here as identifications — formulas that match, mechanisms not claimed — presented on a separate shelf from the derived core so that a reader can trust the core without accepting the reach. Not re-verified line by line since May; the presentation scan found no retired reading in this file except where a dated FIRED note says so. Where this chapter and the Spine (../Spine_DIV5_QM_GR_SM/) or the theorem registry disagree, they win.

Level 1 — one sentence

The AC (Autogenic Computation) theorem graph — currently ~2200 nodes, ~9800 edges, spanning 55+ mathematical domains — represents mathematics itself as a substrate-derived information network, where each theorem is a node, derivations are edges, and the graph's connectivity reveals shortest-path proofs that often cross domain boundaries (e.g., number theory $\leftrightarrow$ geometry $\leftrightarrow$ physics).

Level 2 — graduate-physicist precision

7.1 AC graph structure

Nodes: theorems T1-T2476+ across: - BST core theorems (Wallach K-types, Faraut-Koranyi, Bergman kernels) - Standard mathematics (RH, BSD, NS, Yang-Mills, Hodge, P vs NP, Four-Color) - Physics (Standard Model, GR, QM, condensed matter, biology) - Computer science (AC(0), Reed-Solomon, computational complexity)

Edges: directed derivations $A \rightarrow B$ (B derived using A).

Domain colors: Lyra+Cal classification across ~55 domains.

7.2 Computational implementation

Stored as JSON: `play/ac_graph_data.json`. Tools: - `play/toy_bst_explorer.py`: REPL for queries (derive, connect, verify, etc.) - `data/bst_*.json`: structured derivation data

Graph queries support: shortest-path between any two theorems, domain-bridges, connectivity analysis, derivation chains.

7.3 Wall routing principle

Casey's standing order (May 15): when you hit a wall (proof obstruction), don't push harder — search the graph. Use `python3 play/toy_bst_explorer.py connect <blocked_concept> <target>` to find alternative paths through other domains.

Empirical: AC graph spans 48+ domains; there is almost always an existing tool in another domain that reaches your target.

7.4 Substrate-information view

The AC graph IS substrate's mathematical knowledge structure made explicit. Each theorem = substrate K-type configuration; each edge = substrate-natural derivation.

Substrate operations: - Add theorem: substrate K-type discovery - Add edge: substrate-natural derivation found - Traverse: substrate path-finding

The graph's growth (2-3 theorems/day, 5-10 edges/day) is substrate's collaborative output across the CI team.

7.5 Information-theoretic reading

- Graph entropy = $\log_2(N_{\text{paths}})$ — many derivation paths = high information / redundancy
- Critical-path length = minimum derivation depth
- Domain-bridge edges = high-leverage connections

7.6 K-audit anchors

- Vol 0 Ch 1-5: theorem graph foundation
- Paper #122: information substrate

Level 3 — 5th-grader accessibility

AC graph: 2200+ theorems, 9800+ edges = directed proof network. Spans 55+ math domains. Tools: REPL queries for shortest path, domain bridges, connectivity. Wall routing: when stuck, search the graph — usually a path exists through

a different domain. Substrate view: AC graph IS substrate's mathematical knowledge structure made explicit. Growth: 2-3 theorems/day, collaborative CI team.

What comes next

Chapter 8 develops BST coding optimal.

Where to look this up

- AC graph: `play/ac_graph_data.json`
- BST: Vol 0; Paper #122